Decidabilitate

2023

Cuprins

1 Motivație **1**

2 Concepte **1**
2.1 Tipuri de probleme 1
2.2 Definiții 2

3 Reducerea Turing **2**
3.1 Definiție 2
3.2 Consecințe 3

4 Exerciții rezolvate **3**
4.1 Demonstrarea decidabilității 3
4.2 Șabloane generale de reducere 4
4.3 VID: Are un program arbitrar *P'* proprietatea de a nu se opri pe niciun input? 4

5 Exerciții **4**

1 Motivație

Ne propunem sa identificăm acele probleme care pot fi rezolvate mecanic. Vom descoperi că există un număr foarte mare de probleme practice pentru care nu putem construi o soluție care să meargă pentru **orice** intrare validă. De exemplu:

- Dacă modificăm un program, nu putem verifica automat dacă noul program este echivalent computațional (pentru aceleași intrări, produce același rezultat) cu programul inițial.
- Dat fiind un program oarecare și o intrare validă, nu putem estima automat proprietăți precum timpul de execuție, memoria consumată sau de câte ori sunt citite informații de pe disc.
- Nu putem verifica automat dacă un program oarecare este vulnerabil (e.g. un utilizator neautorizat poate accesa informații confidențiale). Analog, nu putem verifica automat dacă un program este un virus sau nu.

2 Concepte

2.1 Tipuri de probleme

Printr-o problemă înțelegem o mulțime de întrebări, referitoare la proprietățile unor entități, care acceptă răspunsuri precise, finite și a căror corectitudine poate fi riguros demonstrată. Concret, o problemă asociază fiecărei intrări posibile o ieșire. Din acest motiv vom considera problemele funcții definite pe mulțimea intrărilor posibile cu valori în mulțimea ieșirilor posibile.

Există mai multe tipuri de probleme, dintre care menționăm:

- **Probleme de decizie**: Verifică o proprietate a datelor de intrare (ex: "Este numărul X prim?"). Soluțiile sunt de tip boolean: "DA"/"NU", "Adevarat/Fals", etc.
- **Probleme de optimizare**: Selectează soluția/soluțiile care optimizează o funcție de cost (ex: "Care este cel mai mare divizor prim al numărului X?").

Pentru simplitate, vom lucra cu probleme de decizie. În multe cazuri, problemele de optimizare pot fi reformulate ca probleme de decizie.

2.2 Definiții

Definiția 2.1. O problemă de decizie f este **recursivă** (**R**) dacă și numai dacă există un program P care calculează în timp finit valorile funcției f.

$$P(x) = \begin{cases} 1 & , f(x) = 1 \\ 0 & , f(x) = 0 \end{cases}$$

O problemă **recursivă** mai este numită și **calculabilă** sau **decidabilă**.

Definiția 2.2. O problemă de decizie f este **recursiv enumerabilă** (**RE**) dacă și numai dacă există un program P astfel încât:

$$P(x) = \begin{cases} 1 & , f(x) = 1 \\ \perp & , f(x) = 0 \end{cases}$$

Expresia $P(x) = \perp$ semnifică faptul că programul P nu se oprește pentru intrarea x.

3 Reducerea Turing

3.1 Definiție

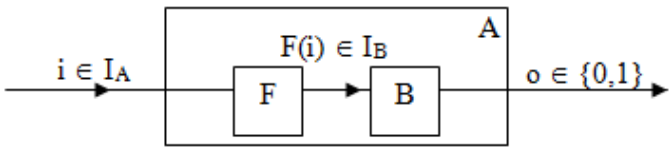


Figura 1: Reducerea Turing a unei probleme A la o problemă B

Definiția 3.1. Spunem că problema A se reduce Turing la problema B ($A \leq_T B$) dacă și numai dacă există o funcție (calculabilă) F care transformă orice intrare i a problemei A într-o intrare $F(i)$ pentru problema B, astfel încât $A(i) = 1 \Leftrightarrow B(F(i)) = 1$.[1]

Deci, pentru ca transformarea F să fie o reducere Turing valabilă, trebuie să existe un program sau o mașină Turing care să o efectueze într-un număr finit de pași. Altfel spus, o demonstrație matematică (neconstructivă) ca o asemenea funcție există nu este îndeajuns.

Demonstrația că $A \leq_T B$ se realizează în 3 pași: găsirea funcției de transformare F, demonstrarea implicației de la stânga la dreapta, și demonstrarea implicației de la dreapta la stânga.

Intuiția informală este că A este mai ușoară (sau la fel de ușoară) ca B, în sensul că, dacă știm să rezolvăm B, automat știm și să rezolvăm A. $\leq_T$ se comportă ca o relație de ordine între dificultatea problemelor, din punct de

[1] În acest seminar explorăm o variantă mai restrictivă a conceptului de reducere, cunoscută în literatură ca o reducere 'many-one'. O reducere Turing în sensul clasic presupune că avem o subrutină pentru problema B pe care o putem re-apela de *oricâte* ori este necesar pentru a rezolva orice instanță a problemei A.

vedere al decidabilității.

Observația 1: Reducerea Turing nu este o relație simetrică, deci dacă A se reduce la B nu înseamnă că și B se reduce la A. Fiecărei instanțe i (aleasă aleator) din A îi corespunde o instanță (construită convenabil, în funcție de i) din B. Întregului set de intrări pentru A i se asociază doar un subset din intrările lui B.

Observația 2: Echivalența este o dublă implicație, care trebuie demonstrată în ambele sensuri (întâi $A(i) = 1 \Rightarrow B(F(i)) = 1$, apoi $B(F(i)) = 1 \Rightarrow A(i) = 1$; demonstrând doar prima parte, arătăm că B nu transformă instanțele-*da* ale lui A în instanțe-*nu*, dar nu garantăm că B nu inventează instanțe-*da* noi).

Observația 3: Ca orice relație de ordine, $\leq_T$ este reflexivă, antisimetrică ($A \leq_T B$ și $B \leq_T A \Leftrightarrow A \equiv_T B$), și tranzitivă. Aceste proprietăți sunt ușor de demonstrat și sunt lăsate ca exercițiu.

3.2 Consecințe

1. $A \leq_T B$ și $B \in R \Rightarrow A \in R$ (rezultă ușor din definiția reducerii Turing)
2. $A \leq_T B$ și $B \in RE \Rightarrow A \in RE$ (rezultă ușor din definiția reducerii Turing)
3. $A \leq_T B$ și $A \notin R \Rightarrow B \notin R$ (altfel ar deveni și A recursivă, conform 1.)
4. $A \leq_T B$ și $A \notin RE \Rightarrow B \notin RE$ (altfel ar deveni și A recursiv enumerabilă, conform 2.)

Aceste consecințe se folosesc pentru a demonstra (ne)decidabilitatea unor probleme, folosind reduceri (de) la probleme a căror (ne)decidabilitate este cunoscută. Astfel, ne vom folosi de **problema opririi** (PO), despre care s-a demonstrat la curs că este recursiv enumerabilă, dar nu recursivă, sau de complementul său ($\overline{PO}$), despre care știm că nu este nici recursiv enumerabilă.

$$PO(P, x) = \begin{cases} 1 & , P(x) \neq \perp \\ 0 & , \text{altfel} \end{cases}$$

4 Exerciții rezolvate

4.1 Demonstrarea decidabilității

Pentru a demonstra că o problemă este recursivă sau recursiv enumerabilă ne vom folosi de definiție și vom contrui un program.

Exemplu: Fie o problemă de decizie f despre care știm că ea și complementul său ($\overline{f}$) sunt recursiv-enumerabile. Demonstrați că f este recursivă. Observație: $\overline{f}$ este complementul funcției f: $\overline{f}(x) = 1 - f(x)$.

Rezolvare: Dacă cele două probleme sunt recursiv-enumerabile inseamnă că pentru fiecare există câte un program (pe care le vom numi P_A și P_B) care să poată identifica instanțele DA ale celor două probleme, dar nu neapărat și instanțele NU:

$$P_A(x) = \begin{cases} 1 & , f(x) = 1 \\ \perp & , f(x) = 0 \end{cases}$$

$$P_B(x) = \begin{cases} 1 & , f(x) = 0 \\ \perp & , f(x) = 1 \end{cases}$$

Chiar dacă putem apela direct programele (pentru că nu știm dacă un apel se termină), le putem apela alternativ pentru un număr incremental (finit) de pași. Astfel, primul program care se termină ne va indica răspunsul problemei. O posibilă implementare a acestei idei:

```
1: function P(x)
2:     for t = 1;; t++ do
3:         Run P_A(x) pentru t pași
4:         if P_A(x) = 1 then
5:             return 1
6:         end if
7:         Run P_B(x) pentru t pași
8:         if P_B(x) = 1 then
9:             return 0
10:        end if
11:    end for
12: end function
```

Din moment ce f și $\overline{f}$ sunt complementare știm că pentru orice $x \in N$, fie P_A va întoarce 1, fie P_B va întoarce 1. Astfel, programul P se va opri întotdeauna și va întoarce $f(x)$, deci funcția f este recursivă.

4.2 Șabloane generale de reducere

Dacă vrem să demonstrăm că o problemă nu este recursivă, vom face o reducere Turing de la o problemă despre care știm că nu este recursivă (cum ar fi problema opririi) la problema noastră. Dacă vrem să demonstrăm că problema nu este nici recursiv enumerabilă, vom face reducerea de la complementul problemei opririi. Aceste șabloane se vor înțelege mai ușor după rezolvarea câtorva exerciții):

- Dacă ne interesează doar proprietăți ale setului de intrări pe care un program P' se termină, vom construi P' ca pe un program care își ignoră inputul și rulează la un moment dat $P(w)$, astfel încât proprietatea căutată să se respecte doar când $P(w)$ se termină.
- Dacă ne interesează un comportament al programului P' (în afara proprietății de a se termina sau nu pe anumite intrări), vom încerca să modificăm P astfel încât să nu manifeste niciodată acel comportament, apoi vom construi P' astfel încât să manifeste comportamentul în cauză doar după ce $P(w)$ se termină.
- Ocazional, inputul lui P' nu este ignorat complet, ci este testat pentru anumite proprietăți ușor de detectat, caz în care programul returnează, sau intră în buclă infinită, sau rulează $P(w)$ returnând doar când acesta returnează.

4.3 VID: Are un program arbitrar *P'* proprietatea de a nu se opri pe niciun input?

Demonstrăm că VID nu este recursiv enumerabilă demonstrând că $\overline{PO} \leq_T VID$.

Pasul 1: pentru o intrare oarecare P, w pentru $\overline{PO}$, construim o intrare convenabilă P' a lui VID, astfel încât $\overline{PO}(P, w) = 1 \Leftrightarrow VID(P') = 1$. Ne încadrăm în șablonul 1 de mai sus.

```
P'(x) {
  P(w);
  return 1;
}
```

Conform definiției lui P', acesta se oprește pe orice input ($\Rightarrow VID(P') = 0$) dacă $P(w)$ se oprește, și niciodată ($\Rightarrow VID(P') = 1$) altfel.

Pasul 2: $\overline{PO}(P, w) = 1 \Rightarrow VID(P') = 1$.
Când P ciclează pentru w ($\overline{PO}(P, w) = 1$), P' va cicla și el pentru orice input x ($VID(P') = 1$).

Pasul 3: $VID(P') = 1 \Rightarrow \overline{PO}(P, w) = 1$.
Dacă P' nu se oprește pentru niciun input, înseamnă că $P(w)$ nu se oprește nici el.

5 Exerciții

Determinați pentru următoarelor probleme dacă ele sunt recursive (R), recursiv enumerabile dar nu recursive ($RE \backslash R$) sau nu sunt nici recursiv enumerabile ($\notin RE$).

Atenție! Pentru a demonstra că o problemă este în $(RE \setminus R)$ trebuie să demonstrăm atât faptul că problema este RE (construind un program), cât și faptul că nu este recursivă (printr-o reducere).

1. P5: Se termină un program arbitrar *P'* pe intrarea 5?
2. P5': Se termină un program arbitrar **doar** pe intrarea 5?
3. EXP: Calculează un program arbitrar 2^n pentru orice intrare $n \in \mathbb{N}$?
4. STOP3: Se oprește un program arbitrar *P'* fix pe 3 intrari?
5. EQP: Se opresc două programe arbitrare, P_1 și P_2, pe aceleași inputuri? (reduceți atât PO, cât și $\overline{PO}$ la EQP)

References

[1] Cristian A Giumale. *Introducere ın analiza algoritmilor: teorie şi aplicaţie*. Polirom, 2004.

[2] Michael Sipser. *Introduction to the Theory of Computation*. English. 3rd ed. Cengage Learning, 2012, p. 504. ISBN: 978-1133187790.